CS 361
Operating Systems — II

T.Y.B.Sc Computer Science | Semester VI | CBCS Pattern

Definitions · Key Concepts · Diagrams · Short Notes · Core Working

---

All 5 Chapters Covered

Chapter 1: Process Deadlocks · Chapter 2: File System Management

Chapter 3: Disk Scheduling · Chapter 4: Distributed OS · Chapter 5: Mobile OS

Ch 1 | Process Deadlocks

What is a Deadlock?

A **deadlock** is a situation where a set of processes are **blocked forever**, each holding a resource and waiting for another resource held by another process — forming a circular chain of waiting.

Example: P1 holds Printer, needs Disk. P2 holds Disk, needs Printer. Neither can proceed → DEADLOCK!

4 Necessary Conditions (Coffman, 1971)

All 4 must hold simultaneously for a deadlock to occur:

#	Condition	Explanation
1	Mutual Exclusion	Only one process can use a resource at a time (non-shareable)
2	Hold & Wait	Process holds ≥1 resource AND waits for more resources held by others
3	No Preemption	Resources cannot be forcefully taken away; only voluntarily released
4	Circular Wait	P0 waits for P1, P1 waits for P2, ... Pn waits for P0 — circular chain

Resource Allocation Graph (RAG)

A **directed graph** used to represent resource allocation status in a system.

- **Request Edge** Pi → Rj : Process Pi is requesting resource Rj (not yet allocated)
- **Assignment Edge** Rj → Pi : Resource Rj is allocated to process Pi
- **No cycle** = No deadlock
- **Cycle + single instance** per resource = Deadlock
- **Cycle + multiple instances** = May or may not be deadlock

3 Methods to Handle Deadlock

Method	When Applied	How
Prevention	Before deadlock	Eliminate at least one of the 4 necessary conditions
Avoidance	Before deadlock	Dynamically check safe/unsafe state before granting each request
Detection	After deadlock	Let deadlock occur; detect using algorithms and recover

Deadlock Prevention — How to Eliminate Each Condition

- **Mutual Exclusion:** Make resources shareable where possible (difficult for printers, tape drives)
- **Hold & Wait:** Process must request ALL required resources before starting execution
- **No Preemption:** If a request fails, preempt all currently held resources; restart when all available
- **Circular Wait:** Assign priority numbers; processes request resources only in increasing priority order

Safe State & Deadlock Avoidance

A state is **safe** if the OS can allocate resources to every process in some sequence without causing deadlock.

- Safe State → No deadlock possible
- Unsafe State → Deadlock is possible (but not certain)
- Deadlock State → Always an unsafe state

Safe Sequence: <P1, P2, ...Pn> where each Pi's remaining needs can be satisfied by currently available resources + resources held by all Pj (j < i).

Banker's Algorithm — Multiple Instances (Dijkstra, 1965)

Named after banking: a bank never allocates cash in a way that it cannot satisfy all customers.

Data Structure	Size	Meaning
Available	1 × m	Available instances of each resource type
Max	n × m	Maximum demand of each process
Allocation	n × m	Currently allocated resources to each process
Need	n × m	Remaining need = Max – Allocation

① **Safety Algorithm:**

- Step 1: Work = Available; Finish[i] = false for all i
- Step 2: Find i where Finish[i] = false AND Need[i] ≤ Work
- Step 3: Work = Work + Allocation[i]; Finish[i] = true → go to Step 2
- Step 4: If all Finish[i] = true → SAFE STATE ✓

② **Resource-Request Algorithm:**

- Step 1: If Request[i] > Need[i] → Error (exceeded maximum claim)
- Step 2: If Request[i] > Available → Process must wait
- Step 3: Pretend to allocate (Available –= Request; Allocation += Request; Need –= Request)
- Step 4: Run Safety Algorithm → If Safe: Grant; If Unsafe: Rollback & Wait

Deadlock Detection

- **Single Instance:** Use Wait-For Graph — remove resource nodes, keep only process nodes. Cycle = Deadlock
- **Multiple Instances:** Algorithm similar to Banker's using Available, Allocation, Request matrices
- If Finish[i] = false after algorithm → Process Pi is deadlocked

Complexity: $O(m \times n^2)$ operations to detect deadlock state

Recovery from Deadlock

Method 1 — Process Termination:

- Kill ALL deadlocked processes (expensive but guaranteed to break deadlock)
- Kill ONE process at a time until deadlock is eliminated (run detection after each kill)

Method 2 — Resource Preemption:

- **Select Victim:** Based on priority, cost, and completion percentage
- **Rollback:** Roll the selected process back to a previously safe state
- **Prevent Starvation:** Ensure same process is not selected as victim repeatedly

Prevention vs Avoidance vs Detection — Differences

Factor	Prevention	Avoidance	Detection
When applied	Before deadlock	Before deadlock	After deadlock
Info needed	None (static rules)	Max future needs (advance)	Current resource state
Overhead	Low	Medium	High (recovery needed)
Resource util.	Poor	Moderate	Good
Algorithm used	Eliminate 1 condition	Banker's Algorithm	Wait-for graph / Detection algo

Ch 2 | File System Management

What is a File?

A **file** is a named collection of related information stored on secondary storage. It is the basic logical storage unit of the OS.

A **File System** is the part of the OS responsible for organizing, managing, and retrieving files. It resides permanently on disk.

File Attributes

Name · **Identifier** · **Type** · **Location** · **Size** · **Protection** (R/W/X) · **Time / Date / User ID**

File Operations

Operation	Description
Create	Find space for new file → add entry in directory
Write	Provide filename + data → update write pointer
Read	Provide filename + memory location → update read pointer
Seek	Move file pointer to specific location (no actual I/O needed)
Delete	Search → release all disk space → erase directory entry
Truncate	Delete file contents; keep attributes; set length to 0
Open	Load file info into open-file table; allows further operations
Close	Save all changes permanently; release all resources

Layered File System Architecture

Layer	Name	Role
Layer 1	Application Programs	User level — calls logical file system
Layer 2	Logical File System (LFS)	Manages directory structure; handles protection & security
Layer 3	File Organization Module	Maps logical blocks to physical disk blocks
Layer 4	Basic File System	Issues read/write commands to I/O drivers
Layer 5	I/O Control (Device Drivers)	Interrupt handlers; transfers data between memory and disk
Layer 6	Devices	Actual physical disk/tape hardware

File Access Methods

Method	How It Works	Best For	Limitation
Sequential	Read/write in order from beginning; uses file pointer	Tape-based; scanning all records	Slow for random access
Direct (Random)	Access block n directly using block number; disk model	Databases; direct lookups	Not suitable for all file types
Indexed (ISAM)	Separate index file maps key → address; binary search on index	Fast search; large sorted files	Extra storage for index

Directory Structures — Comparison

Type	Structure	Advantage	Disadvantage
Single-Level	One directory; all users share it	Simple; easy file operations	No duplicate names; no user separation
Two-Level	MFD (Master) → UFD per user	User isolation; same filename for different users	No sharing between users
Tree-Structured	Hierarchical subdirectories; abs/relative paths	Efficient search; grouping of files	Complex deletion of directories
Acyclic Graph	Tree + shared files via links; no cycles; ref. counting	File sharing; flexible	Complex; dangling pointer on delete
General Graph	Cycles allowed; multiple parent directories	Most flexible; full sharing	Needs garbage collection

File Allocation Methods — Comparison

Method	How It Works	Advantages	Disadvantages	Used In
Contiguous	File occupies consecutive blocks. Dir: (name, start, length)	Fast sequential + direct access; Simple	External fragmentation; must know size in advance	IBM 370
Linked	Linked list of scattered blocks. Dir: (name, start, end). FAT is a variation.	No external fragmentation; file can grow any time	Only sequential access; pointer overhead; reliability problem	MS-DOS (FAT), DEC TOPS-10
Indexed	Index block stores all pointers to file blocks. Dir: (name, index block)	Direct access; no external fragmentation	Overhead for small/large files; complex multilevel indexes	UNIX (inodes), DEC VMS

Free Space Management Methods

Method	How It Works	Key Point
Bit Vector (Bitmap)	1 bit per block; 1 = free, 0 = allocated	Simple, efficient; easy to find contiguous free blocks
Linked List	All free blocks linked together; pointer to first block kept in memory	Not efficient — traversal reads every block
Grouping	First free block stores addresses of next n free blocks	Many free block addresses found quickly
Counting	Store (first free block, count of consecutive free blocks)	Efficient when large contiguous free regions exist
Space Map (ZFS)	Log of free/allocated space per disk region	Designed for Zettabyte-scale file systems (Sun ZFS)

Ch 3 | Disk Scheduling

What is Disk Scheduling?

Technique used by the OS to determine the **order** in which disk I/O requests are serviced, to minimize seek time and maximize throughput.

Key Terms:

- **Seek Time:** Time for disk arm to move head to the required track
- **Rotational Latency:** Time for desired sector to rotate under the read-write head
- **Transfer Time:** Time to actually transfer the data
- **Access Time = Seek Time + Rotational Latency**

Disk Structure

- **Platter** → divided into circular **Tracks** → each track divided into **Sectors**
- **Cylinder:** Set of same tracks on all platters at the same distance from center
- **R/W Head:** Reads and writes data from sectors

All Disk Scheduling Algorithms

Example Queue: 95, 180, 34, 119, 11, 123, 62, 64 | **Head starts at: 50** | Tail (max): 199

Algorithm	How It Works	Total Movement	Advantage	Disadvantage
FCFS	Service requests in exact arrival order: 50→95→180→34→119→11→123→62→64	644 tracks (WORST)	Fair; no starvation	Worst seek time; no optimization
SSTF	Always serve the cylinder nearest to current head position (like SJF scheduling)	236 tracks	Low avg response time; high throughput	Starvation possible; high variance in response
SCAN (Elevator)	Move in one direction serving all requests; reverse direction at disk end	230 tracks	High throughput; low variance	Long wait for locations just visited
C-SCAN	Like SCAN but returns to beginning without servicing on return trip	187 tracks	More uniform wait time than SCAN	Slightly higher total seek time
LOOK	Like SCAN but only goes to last request in each direction (not disk end)	208 tracks	Better than SCAN; no wasted end-of-disk travel	Non-uniform waiting time
C-LOOK	Like C-SCAN but jumps from last request directly to first request	157 tracks (BEST)	Best balance; no end-of-disk waste	Slightly complex

Best choice for default algorithm: C-LOOK or SCAN. SSTF is common but causes starvation. FCFS is the simplest but worst. The disk scheduling algorithm should be a separate OS module so it can be easily replaced.

Ch 4 | Introduction to Distributed Operating Systems

What is a Distributed System?

A collection of **autonomous computers** connected via a communication network, cooperating through **message passing**, appearing as a **single system** to the user.

A **Distributed OS** is system software that manages many computers but presents the interface of a **single computer** to the user.

Design Goals of Distributed Systems

#	Goal	Description
1	Resource Sharing	Share hardware, software, and data across geographically dispersed nodes
2	Openness	Extensible via published software interfaces; standards-based
3	Scalability	Easily expand from small to large number of machines
4	Transparency	Location / replication / failure transparency — hides complexity from user
5	Fault Tolerance	Failure of few nodes does not affect availability of rest of system
6	Concurrency	Multiple tasks execute simultaneously on different machines

Advantages & Disadvantages

Advantages	Disadvantages
Improved reliability & availability (fault tolerant via replication)	Security — both nodes and connections need securing
Modular expandability — add nodes without replacing existing	Very little mature distributed OS software available
Resource sharing across the network	High setup and maintenance cost
Enhanced performance — concurrent execution on multiple machines	Network overloading if all nodes send data at once
Improved efficiency via load distribution	Data loss — messages may be lost over communication network

Architectural Styles

Style	Description	Example
Layered Architecture	Components in layers; each layer talks only to the layer below it; clear hierarchy	OSI model, OS layers
Object-based Architecture	System components are objects; communicate via RPC / method calls; encapsulation	CORBA, Java RMI
Resource-Centered (REST)	Resources identified by URIs; uniform interface for all operations	RESTful Web Services

System Architecture — Organization Types

Type	Description	Key Feature
Centralized	One powerful server serves all clients (Client-Server model)	Simple; single point of failure

Type	Description	Key Feature
Decentralized / P2P	All nodes equal; no central server. Structured P2P uses DHT (Chord); Unstructured uses flooding	No single point of failure; highly scalable
Hybrid	Combination of centralized + P2P. Example: BitTorrent uses central tracker + P2P sharing	Best of both worlds

NFS — Network File System

- Developed by **Sun Microsystems**
- Allows clients to access files on remote servers **as if they were local**
- Uses **RPC (Remote Procedure Calls)** for all client-server communication
- **VFS (Virtual File System)** interface routes operations to local FS or NFS client
- NFS is largely **independent of local file systems** — very portable
- NFSv3 is widely used; NFSv4 is the latest version

Types of Distributed Systems

Type	Description	Example
Cluster Computing	Homogeneous systems connected via LAN for high performance computing	Linux Beowulf clusters
Grid Computing	Heterogeneous, geographically dispersed nodes sharing resources over internet	SETI@home
Cloud Computing	On-demand network access to shared pool of configurable resources	AWS, Google Cloud (SaaS/PaaS/IaaS)

Ch 5 | Mobile Operating Systems

What is a Mobile OS?

A **Mobile OS** (also called mobile platform or handheld OS) is an operating system specifically designed to run on mobile devices — phones, tablets, smartwatches, and PDAs.

It combines PC OS features with mobile-specific features: **touchscreen, camera, GPS, Bluetooth, Wi-Fi, voice recognition, power management**, etc.

Examples: Android (Google) · iOS (Apple) · Symbian (Nokia) · BlackBerry OS (RIM) · Windows Mobile (Microsoft) · Palm OS (Palm Inc.)

Generic Mobile OS Structure (6 Layers)

Layer	Component	Role
Layer 1	Mobile Applications	Customer-level apps — micro-browsers, retail, messaging
Layer 2	GUI (Graphical User Interface)	Limited display; touch interface; less than desktop GUI
Layer 3	API Framework	Bridge between low-level components and application layer
Layer 4a	Multimedia	Image, video, audio functionality
Layer 4b	Communication Infrastructure	TCP/IP, Bluetooth, Wi-Fi, wireless stack
Layer 4c	Security	Wireless networks inherently vulnerable; encryption, auth
Layer 5a	Computer Kernel	Central module; provides essential services to all components
Layer 5b	Power Management	Manages battery; enters low-power sleep modes
Layer 5c	Real-time Kernel	Handles time-critical operations like voice calls
Layer 6	Hardware Controller	Bottom layer; interfaces with physical hardware — display, sensors

Features of Mobile OS

#	Feature	Description
1	Easy to Use	Attractive GUI; simple buttons; interactive and intuitive
2	Good App Store	Large collection of simple, interactive, useful apps
3	Good Battery Life	Power management to prolong battery life; Symbian known for minimal battery demand
4	Data Management	Organized calendars, alarms, reminders; controlled network usage
5	Multitasking	Run multiple apps simultaneously; rapid switching between apps
6	Mobility	Works normally while the device is being carried
7	Connectivity	Connect to Wi-Fi, Bluetooth, USB, other mobile devices and computers
8	Security	Protects data, identity, and availability from wireless threats
9	Open Platform	Supports third-party developers and independent software vendors
10	Innovation	Constantly evolving with mobile-specific features and technologies

Special Constraints & Requirements of Mobile OS

Constraint	Explanation
Limited Memory	Less RAM/ROM than desktops; OS kernel must be very small yet feature-rich
Miniature Keyboard	Touch screen, stylus, or small physical keypad; typing is difficult
Limited Screen Size	Small display; innovative UI needed; less information visible at once
Limited Processing Power	Mostly ARM processors; energy efficient but significantly slower than desktop
Limited Battery Power	Small battery; must enter sleep mode quickly; charging not always possible
Wireless Medium	Must handle mobility and wireless protocols (TCP/IP, Bluetooth, Wi-Fi, USB)

ARM vs Intel Architecture — Power Management

Factor	ARM	Intel x86
Design	RISC — Reduced Instruction Set	CISC — Complex Instruction Set
Power Consumption	Very low (ideal for mobile)	Higher
Performance	Lower for heavy processing	Better for heavy processing
Chip Size	Very small	Larger
Cost	Cheaper	More expensive
Scalability	Less scalable	More scalable
Best Used For	Smartphones, tablets, IoT devices	Desktops, laptops, servers

Commercial Mobile OS — Architecture & Comparison

OS	Developer	Kernel	App Language	Architecture (Layers)	Key Feature
Android	Google / OHA	Linux	Java / Kotlin	Linux Kernel → Android Runtime (DVM) → App Framework → Applications	Open source; most popular; Dalvik VM; SQLite storage
iOS	Apple Inc.	Mach/BSD (XNU)	Swift / Obj-C	Core OS → Core Services → Media → Cocoa Touch → Apps	Most secure; closed source; exclusive to Apple hardware
Windows Mobile	Microsoft	Windows CE	C++ / .NET	Hardware → WinCE OS → Win32 APIs → .NET CF → Apps	MS Office support; PPTP VPN; now discontinued
Symbian OS	Nokia/Ericsson	EPOC (real-time)	EPOC C++, Java	Kernel → Base Services → OS Services → App Services → UI	Real-time multithreaded; data caging; low battery use
BlackBerry OS	RIM	Proprietary	Java (JDE)	Hardware → OS → JVM → BB Platform → Applications	Best enterprise email & security features
Palm OS	Palm Inc.	AMX (ARM 32-bit)	C, C++	HAL → DAL → Core Palm OS → PACE → Applications	Long battery life; Graffiti 2 input; HotSync

Android vs iOS — Key Differences

Factor	Android	iOS
Source Code	Open source (based on Linux)	Closed source (proprietary)

Factor	Android	iOS
Developer	Google / Open Handset Alliance	Apple Inc.
Devices	Multiple manufacturers (Samsung, OnePlus, etc.)	Only Apple devices (iPhone, iPad)
App Language	Java / Kotlin	Swift / Objective-C
App Store	Google Play Store (open submissions)	Apple App Store (strict approval required)
Customization	Highly customizable	Limited customization
Security	Good (Linux kernel-based)	Excellent (sandboxed, signed apps)
OS Updates	Via device manufacturer (can be slow)	Direct from Apple (fast, uniform)
Cost	Free OS; varied device prices	Premium priced; app costs generally higher

---

All the best for your exam! ■ You've got this!